

ROUTEZONE

Prédiction de la gravité des accidents routiers

RAPPORT PROFESSIONNEL — ÉPREUVE E5

Mise en place du monitoring applicatif et résolution d'un incident technique

Candidate	Meriem Abdelouahed
Formation	Développeur en Intelligence Artificielle — Simplon × Microsoft
Certification	RNCP37827
Soutenance	22 juin 2026 — distanciel
Version	1.0 — 29 mai 2026

Sommaire

1. Introduction	3
2. Mon dispositif de monitoring	3
2.1 Vue d'ensemble	3
2.2 Les métriques Prometheus	3
2.3 Dashboard Grafana	4
2.4 Les logs	4
3. Résolution d'incident : les processus fantômes	4
3.1 Détection du problème	4
3.2 Premier diagnostic : tout semble OK	5
3.3 Investigation	5
3.4 L'idée déclic	6
3.5 Résolution	6
3.6 Cause racine	6
3.7 Ce que cet incident m'a appris	6
4. Bilan et perspectives	7

1. Introduction

RouteZone est une application qui permet de prédire la gravité d'un accident de la route grâce à un modèle LightGBM de classification de machine learning. Elle s'appuie sur les données BAAC du Ministère de l'Intérieur et sur les temps d'intervention réels des services de secours, calculés via OSRM.

Le monitoring est un ensemble de techniques qui permettent d'analyser, de contrôler et de surveiller une application en temps réel. En d'autres termes, c'est la surveillance informatique qui mesure la qualité et la fiabilité des logiciels, des réseaux, des serveurs et des équipements.

Un incident est un événement qui peut survenir à n'importe quel moment et causer une perturbation ou un dysfonctionnement de l'application. Sa résolution suit généralement quatre étapes : détection, diagnostic, action, documentation.

Ce rapport présente d'abord mon dispositif de monitoring complet, puis un incident technique concret que j'ai rencontré et résolu pendant le développement du projet.

2. Mon dispositif de monitoring

2.1 Vue d'ensemble

Le monitoring surveille l'application en temps réel après sa mise en service, pour détecter rapidement les anomalies qu'aucun test ne peut anticiper (latence qui grimpe, erreurs, drift du modèle). Il repose sur trois piliers complémentaires :

- **Prometheus** — collecte les métriques en scrapant l'endpoint /metrics de l'API toutes les 15 secondes
- **Grafana** — visualise ces métriques sous forme de graphiques en temps réel
- **Python Logging** — enregistre chronologiquement les événements détaillés dans un fichier texte

Architecture du monitoring RouteZone

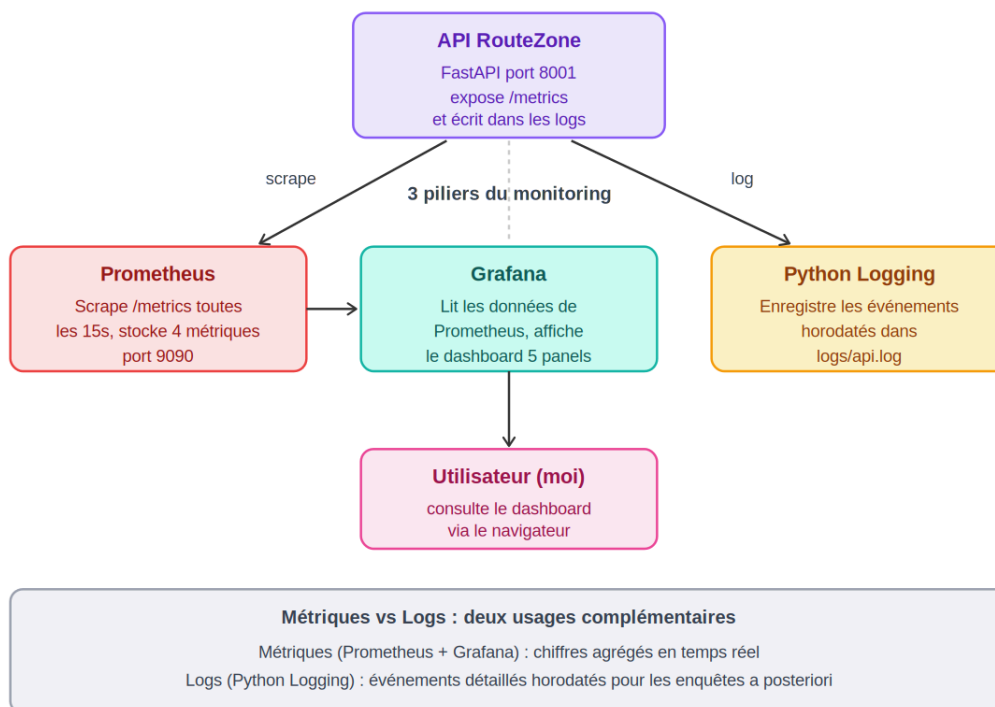


Figure 1 — Architecture du monitoring RouteZone

2.2 Les métriques Prometheus

Quatre métriques sont définies dans metrics.py :

Métrique	Type	Rôle
routezone_predictions_total	Counter	Nombre total de prédictions
routezone_predictions_par_classe_total	Counter	Prédictions par classe (Grave / Pas grave)
routezone_prediction_duration_seconds	Histogram	Latence des prédictions
routezone_errors_total	Counter	Nombre total d'erreurs

Counter : un compteur qui ne fait qu'augmenter (par exemple le nombre total de prédictions depuis le démarrage de l'API).

Histogram : mesure une distribution de valeurs (par exemple, combien de prédictions ont pris moins de 0,1 seconde, combien entre 0,1 et 0,2 seconde, etc.).

2.3 Dashboard Grafana

Grafana lit les données de Prometheus (il ne stocke rien lui-même) et les affiche dans un dashboard de 5 panels :

- Nombre total de prédictions
- Prédictions par classe
- Latence moyenne sur 5 minutes
- Prédictions par minute
- Total des erreurs

Le tableau de bord se rafraîchit en continu et est accessible sur <http://localhost:3000>.

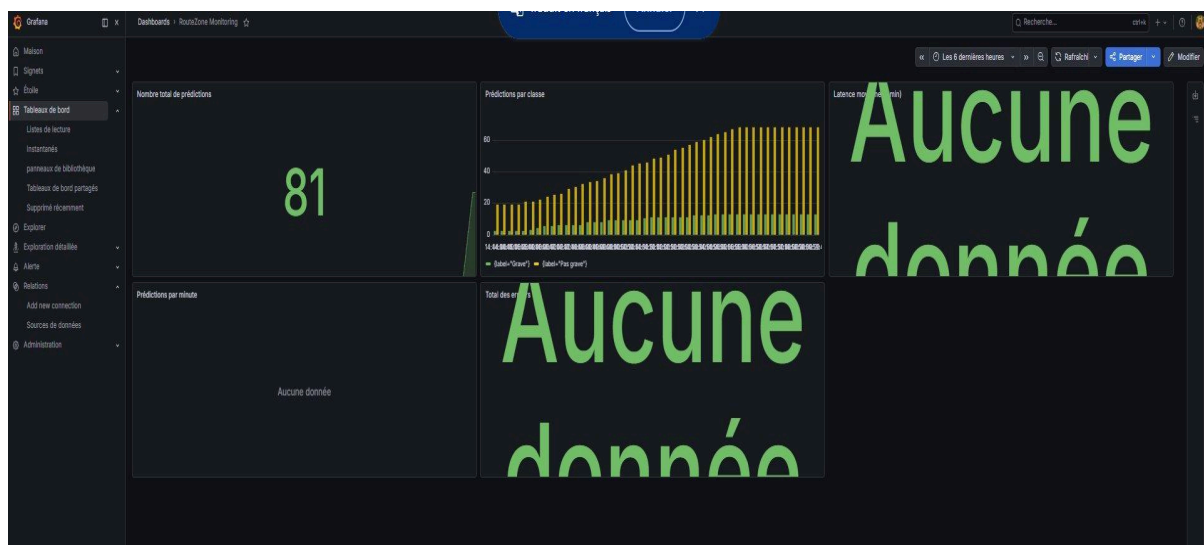


Figure 2 — Dashboard Grafana RouteZone (81 prédictions enregistrées)

2.4 Les logs

En complément des métriques agrégées, le module logger.py écrit chaque événement important (prédiction demandée, prédiction terminée, action RGPD) dans le fichier logs/api.log, avec horodatage et niveau (INFO, WARNING, ERROR). Là où une métrique donne une vue d'ensemble chiffrée, le log fournit la trace précise nécessaire pour enquêter après un incident. Le fichier est persistant et conserve l'historique complet, même après redémarrage de l'API.

```
2026-05-13 14:23:18 | INFO | routezone | Prediction demandee : age=35, vma=80
2026-05-13 14:23:18 | INFO | routezone | Prediction terminee : label=Pas grave, proba=23.4%
2026-05-13 14:25:42 | INFO | routezone | RGPD : suppression predictions pour user_id=3
```

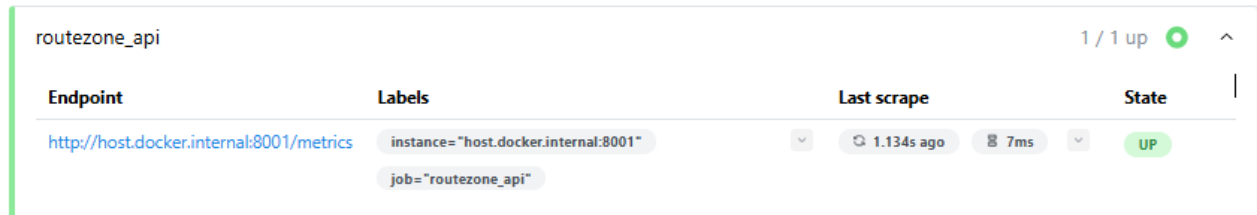
3. Résolution d'incident : les processus fantômes

3.1 Détection du problème

Lors d'une vérification de mon dashboard Grafana, j'ai constaté que tous mes panels affichaient "Aucune donnée" ou des valeurs anciennes, alors que mon API tournait normalement et que je faisais des prédictions de test.

3.2 Premier diagnostic : tout semble OK

J'ai d'abord vérifié l'état de Prometheus sur `http://localhost:9090/targets`. La cible `routezone_api` était affichée en vert (état "EN HAUT"), avec un dernier scrape récent (quelques secondes). À première vue, tout fonctionnait.



Endpoint	Labels	Last scrape	State
http://host.docker.internal:8001/metrics	instance="host.docker.internal:8001" job="routezone_api"	1.134s ago 7ms	UP

Figure 4 — Prometheus targets : cible en vert ("EN HAUT")

Pourtant, quand j'interrogeais directement Prometheus avec la requête `routezone_predictions_total`, il me répondait "Résultat de requête vide".

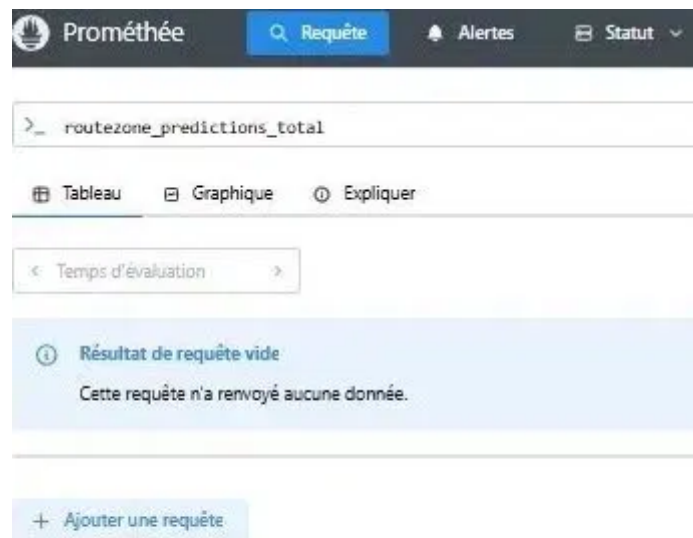


Figure 5 — Prometheus Explorer : "Résultat de requête vide" malgré la cible UP

C'est là que le mystère commençait : l'API exposait bien les métriques (vérifié via `/metrics` dans le navigateur), Prometheus disait qu'il scrapait avec succès, mais aucune donnée n'arrivait dans la base.

3.3 Investigation

J'ai d'abord pensé à un bug Docker. J'ai donc stoppé tous les containers (`docker-compose down`), supprimé le volume Prometheus (`docker volume rm routezone_prometheus_data`), et tout redémarré (`docker-compose up -d`). Le problème persistait.

J'ai ensuite vérifié le fichier `prometheus.yml`, les volumes Docker et l'horloge des containers. Tout semblait correct. Pourtant, Grafana restait vide et Prometheus ne stockait rien.

3.4 L'idée déclic

Je me suis demandée : et si Prometheus allait sur la mauvaise API ? J'ai lancé `netstat -ano | findstr :8001` pour voir tous les processus qui écoutaient sur ce port. J'ai découvert plusieurs PID différents :

TCP	0.0.0.0:8001	LISTENING	9284	(container Docker)	TCP	127.0.0.1:8001	LISTENING
10336	(processus fantôme)	TCP	:::1:8001	LISTENING	10420	(autre processus fantôme)	

Pour confirmer que Prometheus allait sur la mauvaise API, j'ai lancé `docker exec routezone_prometheus wget -qO- http://host.docker.internal:8001/metrics`. En comparant le résultat avec ce que je voyais sur `http://localhost:8001/metrics` depuis le navigateur, j'ai trouvé la preuve définitive : les deux versions de Python étaient différentes (3.11.14 d'un côté, 3.11.9 de l'autre). C'étaient bien deux instances différentes de l'API qui répondaient !

3.5 Résolution

J'ai tué tous les processus Python fantômes :

```
taskkill /PID 10336 /F taskkill /PID 10420 /F Get-Process python3.11 | Stop-Process -Force
```

J'ai vérifié avec `netstat -ano | findstr :8001` qu'il ne restait plus que le PID du container Docker (9284). Puis j'ai relancé le script de prédictions, et Prometheus a immédiatement commencé à enregistrer les métriques.

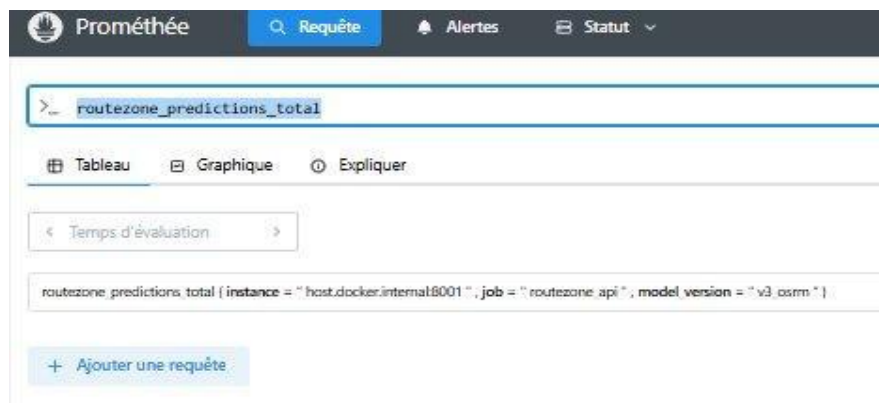


Figure 6 — Prometheus Explorer après résolution : la métrique est trouvée

3.6 Cause racine

Les processus fantômes provenaient d'anciennes sessions d'uvicorn lancées en local avec l'option `--reload`. Cette option redémarre automatiquement le serveur à chaque modification de fichier, ce qui peut laisser des processus orphelins quand on ferme le terminal sans utiliser `Ctrl+C` proprement. Ces processus continuaient à écouter sur `127.0.0.1:8001` et répondaient à Prometheus à la place de mon container Docker, en lui renvoyant un endpoint `/metrics` vide.

3.7 Ce que cet incident m'a appris

- **L'importance de persévérer face à un bug** : les premières solutions classiques (reset Docker, vérification des configs) n'ont rien donné. Sans persévérance, j'aurais abandonné.
- **Connaître ses outils en profondeur** : comprendre que `netstat` permet d'identifier précisément qui écoute sur un port a été décisif.
- **Les processus fantômes existent** : les serveurs lancés en mode `--reload` peuvent survivre à la fermeture de leur terminal.
- **L'importance de la documentation** : j'ai documenté cet incident dans `docs/incidents/incident_processus_fantomés.md`, comme je l'avais déjà fait pour un autre incident technique sur le calibrator du modèle (voir `docs/incidents/incident_calibrator.md`).

4. Bilan et perspectives

Mon dispositif de monitoring fonctionne et m'a déjà servi concrètement, dans le cas de l'incident des processus fantômes décrit ci-dessus, mais aussi pour documenter une autre décision technique (la désactivation du calibrator sur le modèle V3 OSRM).

Mon utilisation du monitoring présente toutefois plusieurs limites que j'assume :

- **Pas d'alertes automatiques** configurées : je dois aller regarder Grafana manuellement. Une amélioration consisterait à configurer Alertmanager pour envoyer un email ou un message Slack en cas de dépassement de seuil.
- **Pas de monitoring du drift** du modèle : je surveille la latence et le nombre de prédictions, mais pas la dérive du modèle dans le temps. Des outils comme Evidently AI seraient à ajouter.
- **Monitoring en local uniquement** : Prometheus, Grafana et l'API tournent sur ma machine. En production, il faudrait des services externalisés (Grafana Cloud, Datadog).

Si je devais aller plus loin, je commencerais par configurer une alerte en cas de chute du Recall ou de pic d'erreurs, puis j'intégrerais un suivi de drift via Evidently AI.